

SOLID Principles

Note: SOLID doesn't appear anywhere in your source material — drafted fresh, since it's one of the most reliably-asked topics in SDE interviews once a candidate has demonstrated basic OOP knowledge. These five principles are guidelines for *designing* class structures well — they build directly on the four pillars, inheritance, and polymorphism covered in earlier topics, so several examples deliberately connect back to those.

Why SOLID Exists

Knowing the four pillars of OOP tells you *what tools* are available (encapsulation, inheritance, polymorphism, abstraction). SOLID is a set of five guidelines for *how to use those tools well* — each principle is a direct response to a specific, recurring way that OOP code goes wrong even when it "technically" uses classes, inheritance, and interfaces correctly.

```
S – Single Responsibility Principle
O – Open/Closed Principle
L – Liskov Substitution Principle
I – Interface Segregation Principle
D – Dependency Inversion Principle
```

S — Single Responsibility Principle (SRP)

A class should have only one reason to change.

Violation:

```
class Employee {
    double calculatePay() { /* payroll logic */ }
    void saveToDatabase() { /* database logic */ }
    void generateReport() { /* reporting/formatting logic */ }
}
```

This `Employee` class has **three** reasons to change: a change in payroll rules, a change in the database schema/technology, or a change in report formatting — each completely unrelated to the others, yet all bundled into one class.

Fix:

```
class Employee { double calculatePay() { /* ... */ } }
class EmployeeRepository { void save(Employee e) { /* ... */ } }
class EmployeeReportGenerator { void generate(Employee e) { /* ... */ } }
```

Why this matters, concretely: if payroll logic and database logic live in the same class, a change to *how data is saved* (e.g., switching databases) risks introducing bugs into completely unrelated payroll calculations, just because they happen to share a file/class — and two developers working on unrelated changes (one on reporting, one on persistence) end up touching the same class, creating unnecessary merge conflicts and review overhead for changes that have nothing to do with each other.

Interview soundbite: "SRP isn't 'a class should only have one method' — it's 'a class should only have one *axis of change*.' A class can have many methods, as long as they're all cohesively serving the same single responsibility, and would all need to change together for the same underlying reason."

O — Open/Closed Principle (OCP)

Software entities should be open for extension, but closed for modification.

Violation:

```
class AreaCalculator {
    double calculate(Object shape) {
        if (shape instanceof Square) return ((Square)shape).side * ((Square)shape).side;
        else if (shape instanceof Circle) return Math.PI * ((Circle)shape).radius * ((Circle)shape).radius;
        // adding a Triangle means coming back HERE and adding another `else if`
    }
}
```

Every new shape type requires **modifying** `AreaCalculator`'s existing, already-tested code — risking breaking what already worked, just to add something new.

Fix — exactly the polymorphic pattern from the Polymorphism topic:

```
interface Shape { double area(); }
class Square implements Shape { double side; public double area() { return side * side; } }
class Circle implements Shape { double radius; public double area() { return Math.PI * radius * radius; } }

class AreaCalculator {
    double calculate(Shape shape) { return shape.area(); } // NEVER needs to change again
}
```

Adding a `Triangle` now means writing a **new** class — `AreaCalculator` is never touched again, ever, for this reason.

Interview soundbite: "This is the direct payoff of the `Shapes* shapes[]` polymorphism example from earlier — OCP is just naming *why* that pattern is good design: extension (new shapes) shouldn't require modification (editing the loop/calculator that already works)."

L — Liskov Substitution Principle (LSP)

Objects of a derived class must be substitutable for objects of the base class, without altering the correctness of the program.

Violation — the classic example:

```
class Rectangle {
    protected double width, height;
    void setWidth(double w) { width = w; }
    void setHeight(double h) { height = h; }
    double area() { return width * height; }
}
class Square extends Rectangle { // "a square IS a rectangle" — seems reasonable!
    @Override void setWidth(double w) { width = w; height = w; } // must keep both sides equal
    @Override void setHeight(double h) { width = h; height = h; }
}

void resize(Rectangle r) {
    r.setWidth(5); r.setHeight(10);
    assert r.area() == 50; // PASSES for a real Rectangle... FAILS for a Square (area would be 100)!
}
```

Even though "a square is-a rectangle" is geometrically true, **Square** **cannot honor the behavioral contract** **Rectangle** establishes (that width and height can be set independently) — substituting a **Square** where a **Rectangle** is expected **breaks** code that correctly relied on that contract.

The exact same failure shape as the Duck scenario from the Abstract Classes topic: forcing **RemoteControlDuck** to implement **fly()** (when it structurally cannot fly meaningfully) is the identical mistake — a subtype that can't honestly fulfill its supertype's contract, forced into the hierarchy anyway because the *conceptual* is-a relationship seemed to fit.

Fix: don't force the inheritance relationship where the *behavioral* contract doesn't actually hold — model **Square** and **Rectangle** as separate types (perhaps both implementing a common **Shape** interface with just **area()**), rather than one inheriting from the other.

Interview soundbite: "LSP is the reminder that 'is-a' in OOP has to mean 'behaves exactly like, from the caller's perspective' — not just 'is-a' in the everyday English/geometric sense. If substituting a subtype can silently break correct code written against the supertype's contract, the inheritance relationship is wrong, no matter how natural it sounds in plain language."

I — Interface Segregation Principle (ISP)

No client should be forced to depend on methods it does not use — prefer many small, specific interfaces over one large, general-purpose one.

Violation:

```
interface Worker {
    void work();
    void eat();
}
class HumanWorker implements Worker {
    public void work() { /* ... */ }
    public void eat() { /* ... */ } // makes sense
}
class RobotWorker implements Worker {
    public void work() { /* ... */ }
    public void eat() { throw new UnsupportedOperationException(); } // forced to implement something meaningless
}
```

`RobotWorker` is **forced** to provide an `eat()` implementation purely because it implements `Worker` — even though eating is meaningless for a robot. This is the **exact same structural mistake** as the `RemoteControlDuck` / `fly()` problem from the Abstract Classes topic.

Fix:

```
interface Workable { void work(); }
interface Eatable { void eat(); }
class HumanWorker implements Workable, Eatable { /* both */ }
class RobotWorker implements Workable { /* only work() – never asked to fake eat() */ }
```

Interview soundbite: "ISP is really LSP's sibling principle, viewed from the interface-design side rather than the inheritance side: LSP says 'don't inherit from a supertype you can't honestly fulfill'; ISP says 'don't design an interface so broad that some legitimate implementers are *forced* into dishonestly fulfilling part of it.' Splitting `Quackable` / `Flyable` in the earlier Duck example was ISP in action, even before this topic named it."

D — Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules; both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions.

Violation:

```
class MySQLDatabase { void save(String data) { /* MySQL-specific code */ } }
class UserService {
    private MySQLDatabase db = new MySQLDatabase(); // directly depends on a CONCRETE, specific class
    void createUser(String data) { db.save(data); }
}
```

`UserService` (a high-level, business-logic module) is **directly tied** to `MySQLDatabase` (a low-level, specific implementation detail). Switching databases, or writing a test with a fake database, requires **modifying** `UserService` **itself**.

Fix:

```

interface Database { void save(String data); } // the ABSTRACTION
class MySQLDatabase implements Database { public void save(String data) { /* ... */ } }
class MongoDBDatabase implements Database { public void save(String data) { /* ... */ } }

class UserService {
    private Database db; // depends on the ABSTRACTION, not a concrete class
    UserService(Database db) { this.db = db; } // the specific implementation is "injected" from outside
    void createUser(String data) { db.save(data); }
}

```

`UserService` now depends only on the `Database` **interface** — it works identically whether it's handed a `MySQLDatabase`, a `MongoDatabase`, or (crucially, for testing) a fake/mock `Database` implementation, **without ever being modified itself**.

This pattern — passing in a dependency from outside, rather than a class constructing its own — is called Dependency Injection, and it's the most common practical technique for actually achieving DIP. It's also exactly why frameworks like Spring (Java) are built around "injecting" dependencies rather than letting classes instantiate their own collaborators directly.

Interview soundbite: "'Inversion' refers to *who depends on whom*: normally you'd think the high-level business logic depends on the low-level database detail. DIP inverts this — both the high-level logic *and* the low-level detail instead depend on a shared abstraction (the interface) sitting between them, so the high-level code no longer cares which specific low-level implementation it's actually talking to."

How the Five Principles Reinforce Each Other

Principle	What it protects against
SRP	A class doing too many unrelated things, coupling unrelated reasons to change
OCP	Adding new behavior requiring risky edits to already-working code
LSP	A subtype that lies about honoring its supertype's contract
ISP	An interface so broad that some implementers are forced to fake unsupported behavior
DIP	High-level logic becoming rigidly welded to specific low-level implementation details

Interview soundbite: "SOLID isn't five unrelated rules — LSP and ISP are the same underlying idea (don't force a fake 'is-a'/'can-do' relationship) applied to inheritance and interfaces respectively. OCP is what you get for free once polymorphism and good abstractions (the outcome of following LSP/ISP/DIP) are in place. SRP and DIP are both, at heart, about controlling *coupling* — SRP within a class's own responsibilities, DIP between layers of a system."

Interview Questions With Answers

Q1. Explain SRP using a concrete example, and clarify the common misconception that it means "one method per class."

Answer: SRP means a class should have exactly one reason to change — one cohesive responsibility — not that it should be limited to a single method. An `Employee` class combining payroll calculation, database persistence, and report generation has three unrelated reasons to change (a payroll rule change, a database migration, a report format change), each of which could require modifying the same class for entirely unrelated purposes — this is what SRP forbids. A class can legitimately have many methods, as long as all of them serve one single, cohesive responsibility that would only ever need to change for one underlying reason.

Q2. How does the Open/Closed Principle connect to the polymorphism concepts covered earlier (e.g., the `Shapes[]` array example)?

Answer: OCP names the actual design benefit that polymorphic code delivers: a loop or function written against a shared interface/base type (like `Shape`) never needs to be modified when a new implementing type (like a new `Triangle` or `Circle`) is added — it's "closed" to modification, since it already works correctly for any type honoring the interface, while the system as a whole is "open" to extension, since new types can be added freely. The alternative (an `instanceof`-based chain of `if/else` checking every concrete type by hand) violates OCP directly, since adding a new type requires going back and editing that existing, already-tested chain.

Q3. Using the classic Square/Rectangle example, explain precisely why making `Square` extend `Rectangle` violates LSP, even though a square genuinely is a kind of rectangle geometrically.

Answer: `Rectangle` establishes a behavioral contract that width and height can be set independently of each other, and that `area()` will reflect whatever the two independently-set dimensions are. `Square`, to remain geometrically valid, must force both dimensions to stay equal whenever either is set — meaning code that correctly relies on `Rectangle`'s contract (e.g., "set width to 5, set height to 10, expect area 50") silently produces a wrong result (area 100) if a `Square` is substituted in. LSP is about behavioral substitutability from the *caller's* perspective, not conceptual/geometric truth — even though "square is-a rectangle" is true in ordinary language, `Square` cannot honestly fulfill the specific

behavioral contract `Rectangle` establishes, which is exactly what LSP requires for a valid "is-a" relationship in OOP.

Q4. How is the Interface Segregation Principle essentially the same underlying idea as the Duck/Quackable/Flyable scenario discussed in the Abstract Classes topic?

Answer: Both scenarios involve an interface (or abstract class contract) that's broader than what every legitimate implementer can honestly support — forcing an implementer like `RobotWorker` (can't eat) or `RemoteControlDuck` (can't fly) to provide a fake, meaningless, or exception-throwing implementation just to satisfy the interface's full method list. ISP's fix — splitting one broad interface into several smaller, more specific ones (`Workable` / `Eatable` , or `Quackable` / `Flyable`) — lets each implementer implement only the specific capabilities it genuinely supports, with the type system accurately reflecting what's actually possible for each type, rather than lying about a capability it doesn't really have.

Q5. What does "dependency injection" mean, and how does it relate to achieving the Dependency Inversion Principle?

Answer: Dependency injection means a class receives its dependencies (collaborating objects it needs to do its job) from outside — typically via constructor parameters — rather than constructing those dependencies itself internally. This is the primary practical technique for achieving DIP: instead of `UserService` directly instantiating a concrete `MySQLDatabase` inside itself (tightly coupling it to that specific implementation), `UserService` instead accepts any `Database` (an abstraction/interface) via its constructor, and whatever concrete implementation is actually needed gets "injected" from outside at construction time — this is exactly what lets `UserService` remain unmodified whether it's used with a real MySQL database, a different database entirely, or a fake/mock implementation for testing.

Q6. Why is "high-level modules should depend on abstractions, not low-level modules" called an *inversion*? What's being inverted?

Answer: In a naive design, the natural direction of dependency is that high-level business logic (like `UserService`) directly depends on the specific low-level implementation detail it needs (like `MySQLDatabase`) — the high-level code reaches "down" to a concrete low-level class. DIP inverts this relationship: instead, an abstraction (an interface like `Database`) is introduced, and *both* the high-level module and the low-level implementation depend on that shared abstraction — the high-level module depends "upward" on the interface, and the low-level implementation also depends "upward" on (implements) the same interface, rather than the high-level module depending directly on the low-level detail. The direction of the high-level module's dependency has been inverted from "depends on a concrete low-level detail" to "depends on an abstraction that the low-level detail happens to implement."

Q7. Why do SRP and DIP both get described as being fundamentally about "coupling," despite operating at different scales?

Answer: SRP controls coupling *within* a single class — ensuring a class's internal responsibilities aren't unnecessarily entangled with each other, so a change to one concern (e.g., persistence) doesn't

risk breaking or require touching an unrelated concern (e.g., payroll logic) just because they happen to share a class. DIP controls coupling *between* layers of a system — ensuring high-level business logic isn't unnecessarily entangled with the specific implementation details of the low-level components it relies on, so a change to (or replacement of) a low-level detail (like swapping databases) doesn't ripple upward and require changes to high-level logic that shouldn't need to care. Both principles are ultimately about minimizing how much unrelated code is forced to change together — just applied at different structural scales (within a class vs. across architectural layers).

Q8. Scenario: A payment processing system has a class `OrderProcessor` that directly instantiates and calls a concrete `PayPalPaymentGateway` class inside its `checkout()` method. The company now needs to also support Stripe as a payment option, and wants to add more providers easily in the future without risky changes to `OrderProcessor`. Using at least two SOLID principles, describe the redesign you'd recommend.

Answer: This calls for both **Dependency Inversion** and **Open/Closed** together. First, introduce a `PaymentGateway` interface (the shared abstraction) with a method like `processPayment(amount)`, and have both `PayPalPaymentGateway` and a new `StripePaymentGateway` implement it — `OrderProcessor` should then depend only on the `PaymentGateway` interface, receiving the specific implementation via constructor injection (dependency injection) rather than instantiating `PayPalPaymentGateway` directly inside itself — this is the Dependency Inversion fix, decoupling `OrderProcessor`'s high-level checkout logic from any specific payment provider's implementation details. Once this is in place, the Open/Closed Principle is satisfied as a direct consequence: adding a third provider later (e.g., `RazorpayPaymentGateway`) just means writing a new class implementing `PaymentGateway` — `OrderProcessor`'s existing, already-tested `checkout()` logic never needs to be touched again, satisfying the company's stated goal of adding future providers without risky changes to the existing processor.